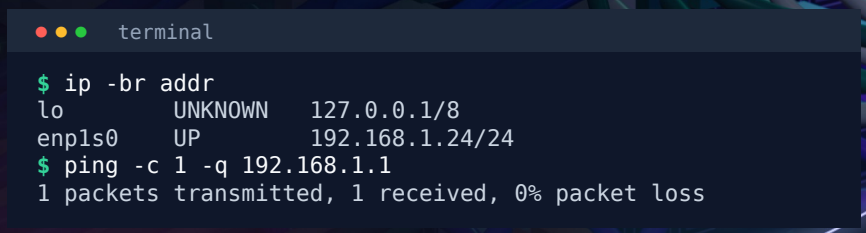
GUIDE PRATIQUE · ADMINISTRATION LINUX

Les 22 commandes réseau Linux

essentielles pour l'administrateur système

Configuration, diagnostic, DNS, capture de paquets, surveillance et pare-feu : la référence pratique pour administrer le réseau sous Linux, avec exemples commentés et bonnes pratiques.



RÉALISÉ PAR

Ngouye GNING

VERSION
1.0

DATE
Octobre 2026

CONTENU
22 fiches · 6 familles · annexes

NAVIGATION

Sommaire

Introduction 3

Vue d'ensemble des 22 commandes 5

Configuration, dépannage et débogage 7

01 ifconfig — Afficher et configurer les interfaces 8
02 ip — La boîte à outils réseau moderne 9
03 ifup · ifdown · ifquery — Piloter les interfaces déclarées 11
04 ethtool — Inspecter et régler la carte réseau 12
05 ping — Tester la connectivité 13
06 traceroute — Tracer le chemin des paquets 14
07 mtr — Diagnostic réseau en temps réel 15
08 route — Gérer la table de routage 16
09 nmcli — Piloter NetworkManager 17

Analyse du réseau et des performances 18

10 netstat — Connexions, ports et statistiques 19
11 ss — Statistiques des sockets 20
12 nc (netcat) — Le couteau suisse du réseau 21
13 nmap — Cartographier le réseau et auditer les ports 22

Résolution DNS 23

14 host — Résolution DNS rapide 24
15 dig — L'outil DNS de référence 25
16 nslookup — Interroger le DNS en mode direct ou interactif 26

Capture et analyse de paquets 27

17 tcpdump — Capturer les paquets en console 28
18 Wireshark — Analyser les protocoles en mode graphique 29

Surveillance du trafic 30

19 bmon — Surveiller la bande passante en temps réel 31

Gestion du pare-feu 32

20 iptables — Le pare-feu Netfilter historique 33
21 firewalld — Le pare-feu dynamique par zones 34
22 ufw — Le pare-feu simplifié 35

Conclusion 36

Annexes 37

A. Aide-mémoire : « Je veux… » 37
B. Correspondance net-tools → iproute2 38
C. Méthode de diagnostic en 8 étapes 38
D. Installation des outils 39
E. Bonnes pratiques et sécurité 40
F. Sources et références 40
G. Glossaire 41

AVANT-PROPOS

Introduction

L'administrateur système est en première ligne pour configurer, maintenir, dépanner et sécuriser les serveurs et les réseaux d'une infrastructure. Sous Linux, l'essentiel de ce travail s'effectue en ligne de commande : rapide, scriptable et disponible même sur un serveur sans interface graphique, la console reste l'outil de référence.



Ce guide présente **22 commandes et utilitaires réseau incontournables**, regroupés en six familles : configuration et dépannage, analyse et performance, résolution DNS, capture de paquets, surveillance du trafic et gestion du pare-feu. Pour chacune, vous trouverez une fiche d'identité, une explication de son rôle, des exemples commentés et les options les plus utiles au quotidien.

Objectifs du guide

- Disposer d'une **référence claire et rapide** à consulter au quotidien ;
- Comprendre **à quoi sert chaque outil** et dans quelle situation l'utiliser ;
- Adopter les **commandes modernes** (`ip`, `ss`, `nmcli`, nftables) tout en sachant lire les outils historiques ;
- Mettre en œuvre une **méthode de diagnostic réseau** structurée et reproductible.

Public visé

Administrateurs systèmes et réseaux, ingénieurs d'exploitation (DevOps, SRE), techniciens support, ainsi que les étudiants qui préparent une certification Linux (LPIC-1, RHCSA, CompTIA Linux+) ou une formation en réseaux et télécommunications.

Conventions utilisées

NOTATION	SIGNIFICATION
`$ commande`	Commande exécutée par un utilisateur standard
`$ sudo commande`	Commande nécessitant les privilèges administrateur (root)
`enp1s0`, `eth0`	Noms d'interfaces : adaptez-les à votre système (`ip -br link`)
`<valeur>`	Paramètre à remplacer par votre propre valeur
`# texte`	Commentaire explicatif dans un bloc de code

À PROPOS DES EXEMPLES

Les sorties affichées sont des exemples illustratifs : elles varient selon la distribution, la version des outils et la configuration du système. Les adresses utilisées (192.168.1.0/24, 198.51.100.0/24, 203.0.113.0/24, example.com) sont des valeurs réservées à la documentation.

SYNTHÈSE

Vue d’ensemble des 22 commandes

Le tableau ci-dessous résume le rôle de chaque outil et son statut actuel. Les outils marqués « Obsolète » ou « Hérité » restent présents sur de nombreux systèmes : il faut savoir les lire, mais privilégier leurs remplaçants modernes pour tout nouveau travail.

N°	COMMANDE	FAMILLE	USAGE PRINCIPAL	STATUT
01	`ifconfig`	Configuration	Afficher et configurer les interfaces	Obsolète
02	`ip`	Configuration	Adresses, liens, routes et voisins	Recommandé
03	`ifup · ifdown · ifquery`	Configuration	Activer, désactiver, interroger une interface	Selon distribution
04	`ethtool`	Configuration	Paramètres de la carte réseau et du pilote	Actif
05	`ping`	Dépannage	Tester la connectivité (ICMP)	Actif
06	`traceroute`	Dépannage	Tracer le chemin vers un hôte	Actif
07	`mtr`	Dépannage	Ping et traceroute en temps réel	Actif
08	`route`	Configuration	Afficher et modifier la table de routage	Obsolète
09	`nmcli`	Configuration	Piloter NetworkManager	Recommandé
10	`netstat`	Analyse	Connexions, ports et statistiques	Obsolète
11	`ss`	Analyse	Statistiques des sockets	Recommandé
12	`nc (netcat)`	Analyse	Tester des ports, transférer des données	Actif
13	`nmap`	Analyse · Sécurité	Découverte d’hôtes et scan de ports	Actif
14	`host`	DNS	Résolution DNS simple	Actif
15	`dig`	DNS	Interrogation DNS détaillée	Recommandé
16	`nslookup`	DNS	Interrogation DNS directe ou interactive	Actif
17	`tcpdump`	Capture	Capture de paquets en console	Actif
18	`Wireshark`	Capture	Analyse graphique des protocoles	Actif
19	`bmon`	Surveillance	Bande passante en temps réel	Actif
20	`iptables`	Pare-feu	Règles Netfilter (filtrage, NAT)	Hérité
21	`firewalld`	Pare-feu	Pare-feu dynamique par zones	Actif
22	`ufw`	Pare-feu	Pare-feu simplifié	Actif

Recommandé à privilégier Actif courant et maintenu Obsolète / Hérité : remplacé par un outil récent Selon distribution disponibilité variable

Où agit chaque outil ?

Les outils réseau s'organisent naturellement selon les couches du modèle OSI. Repérer la couche concernée par un incident permet de choisir immédiatement le bon outil — et d'éviter de chercher une panne DNS avec `ethtool` ou un câble débranché avec `dig`.

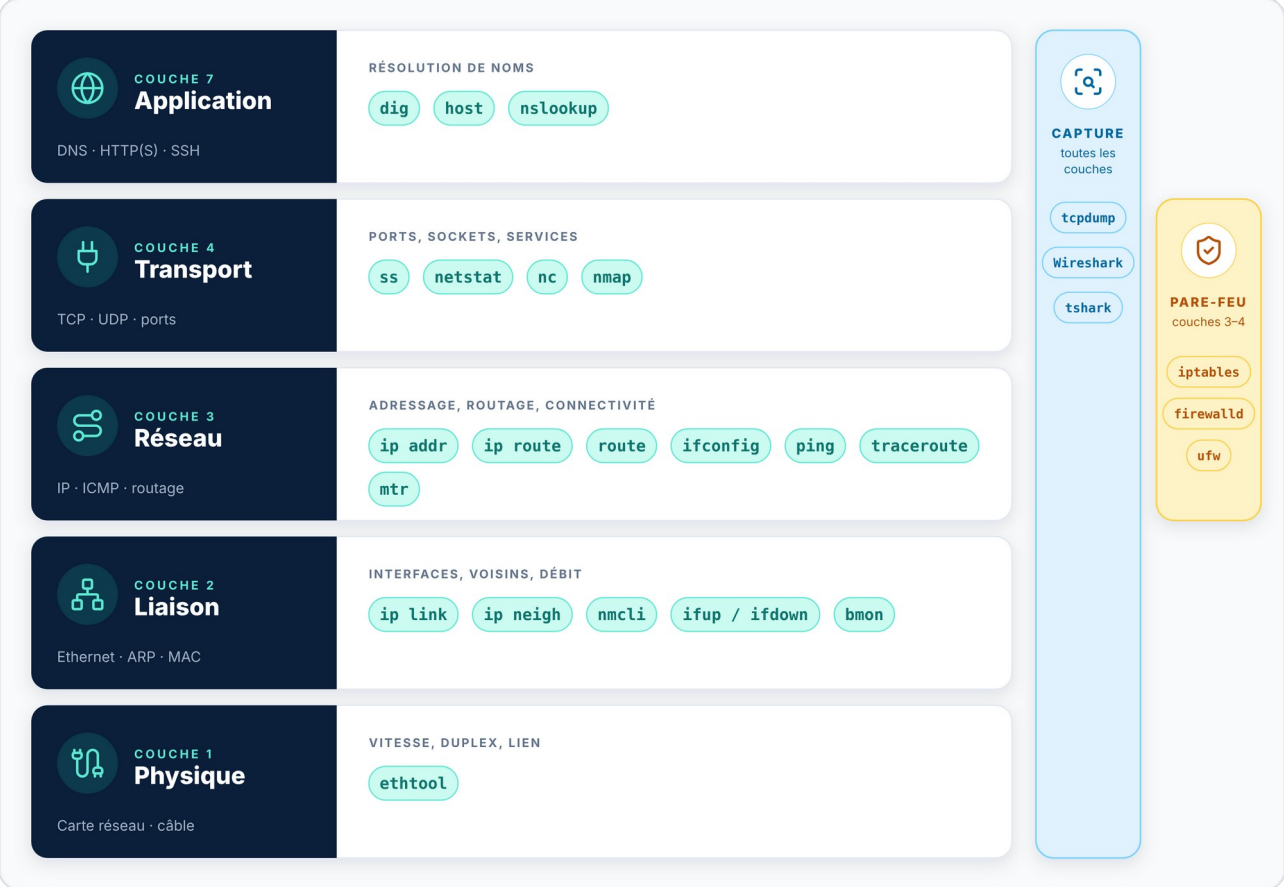


Figure 1 — *Cartographie des 22 outils selon les couches réseau. La capture traverse toutes les couches ; le pare-feu agit sur les couches 3 et 4.*

LECTURE DU SCHÉMA

De bas en haut : on vérifie d'abord le lien physique (`ethtool`), puis l'adressage et le routage (`ip`, `ping`), les ports (`ss`, `nmap`) et enfin les services applicatifs comme le DNS (`dig`). Cette progression est détaillée dans l'annexe C.



Configuration, dépannage et débogage

Ces outils permettent de configurer les interfaces, de vérifier la connectivité et de comprendre le chemin emprunté par les paquets. Ce sont les premiers réflexes de tout diagnostic réseau, du poste de travail au serveur de production.



DANS CETTE PARTIE

01	`ifconfig`	Afficher et configurer les interfaces
02	`ip`	Adresses, liens, routes et voisins
03	`ifup · ifdown · ifquery`	Activer, désactiver, interroger une interface
04	`ethtool`	Paramètres de la carte réseau et du pilote
05	`ping`	Tester la connectivité (ICMP)
06	`traceroute`	Tracer le chemin vers un hôte
07	`mtr`	Ping et traceroute en temps réel
08	`route`	Afficher et modifier la table de routage
09	`nmcli`	Piloter NetworkManager

COMMANDE 01 · CONFIGURATION

01 ifconfig — Afficher et configurer les interfaces

PAQUET	PRIVILÈGES	STATUT
net-tools	Lecture : utilisateur · Modification : root	Obsolète

`ifconfig` (*interface configurator*) est l'outil historique de configuration des interfaces réseau. Il affiche l'état des interfaces actives — adresses IPv4 et IPv6, adresse MAC, MTU (unité de transmission maximale), compteurs de paquets — et permet aussi d'attribuer une adresse IP ou d'activer et désactiver une interface à la demande.

Afficher les interfaces

```bash
# Interfaces actives uniquement
$ ifconfig
# Toutes les interfaces, y compris celles qui sont désactivées
$ ifconfig -a
# Une interface précise
$ ifconfig enp1s0
enp1s0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.24  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 fe80::5054:ff:fe3a:1c2b  prefixlen 64  scopeid 0x20<link>
        ether 52:54:00:3a:1c:2b  txqueuelen 1000  (Ethernet)
        RX packets 169854  bytes 174146270 (166.0 MiB)
        TX packets 125995  bytes 21062129 (20.0 MiB)
```

Lecture de la sortie : `flags` indique l'état de l'interface (UP, RUNNING), `inet` l'adresse IPv4 et son masque, `ether` l'adresse MAC, `mtu` la taille maximale des trames ; les compteurs RX et TX donnent le volume reçu et émis.

Configurer une interface

```bash
# Attribuer une adresse IP et un masque
$ sudo ifconfig eth0 192.168.56.5 netmask 255.255.255.0
# Activer puis désactiver l'interface
$ sudo ifconfig eth0 up
$ sudo ifconfig eth0 down
# Modifier la MTU (trames jumbo)
$ sudo ifconfig eth0 mtu 9000
```

> **OUTIL OBSOLÈTE**
> `ifconfig` appartient au paquet net-tools, qui n'évolue plus et n'est souvent plus installé par défaut. Préférez la commande `ip` (fiche 02). Les modifications ne sont pas persistantes : elles disparaissent au redémarrage.

COMMANDE 02 · CONFIGURATION

02 ip — La boîte à outils réseau moderne

PAQUET	PRIVILÈGES	STATUT
iproute2 (Debian/Ubuntu) · iproute (RHEL/Fedora)	Lecture : utilisateur · Modification : root	Recommandé

`ip`, issue de la suite iproute2, est l'outil de référence pour afficher et manipuler les adresses, les interfaces (liens), le routage et la table des voisins. Elle remplace à elle seule `ifconfig`, `route` et `arp`. Sa syntaxe suit toujours le même modèle : `ip [options] objet commande`.

OBJET	RÔLE	EXEMPLE
`addr` (a)	Adresses IPv4 et IPv6	`ip addr show`
`link` (l)	Interfaces physiques et virtuelles	`ip link show`
`route` (r)	Table de routage	`ip route show`
`neigh` (n)	Table des voisins (ARP / NDP)	`ip neigh show`

Afficher les adresses

```bash
# Vue synthétique : une ligne par interface
$ ip -br addr
lo        UNKNOWN   127.0.0.1/8 ::1/128
enp1s0    UP        192.168.1.24/24 fe80::5054:ff:fe3a:1c2b/64
wlp2s0    DOWN
# Vue détaillée d'une interface
$ ip addr show enp1s0
2: enp1s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
    link/ether 52:54:00:3a:1c:2b brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.24/24 brd 192.168.1.255 scope global dynamic enp1s0
       valid_lft 85912sec preferred_lft 85912sec
```

Gérer les adresses et les interfaces

```bash
# Ajouter puis supprimer une adresse (temporaire)
$ sudo ip addr add 192.168.56.10/24 dev eth0
$ sudo ip addr del 192.168.56.10/24 dev eth0
# Activer / désactiver une interface
$ sudo ip link set eth0 up
$ sudo ip link set eth0 down
# Statistiques détaillées (paquets, erreurs, pertes)
$ ip -s link show eth0
```

Routes et voisins

```bash
$ ip route show
default via 192.168.1.1 dev enp1s0 proto dhcp metric 100
192.168.1.0/24 dev enp1s0 proto kernel scope link src 192.168.1.24
# Quelle route sera utilisée pour joindre cette destination ?
$ ip route get 8.8.8.8
# Table des voisins (équivalent moderne de arp)
$ ip neigh show
192.168.1.1 dev enp1s0 lladdr 10:fe:ed:3d:f3:82 REACHABLE
```

ASTUCE

Les objets s'abrègent : `ip a`, `ip r`, `ip n`. L'option `-c` colore la sortie (`ip -c a`) et `-j` produit du JSON, idéal pour les scripts et l'automatisation.

PERSISTANCE

Les changements effectués avec `ip` sont perdus au redémarrage. Pour une configuration durable, utilisez NetworkManager (`nmcli`), Netplan sur Ubuntu ou le fichier `/etc/network/interfaces` sur Debian.

COMMANDE 03 · CONFIGURATION

03 ifup • ifdown • ifquery — Piloter les interfaces déclarées

PAQUET	PRIVILÈGES	STATUT
ifupdown (Debian/Ubuntu)	root (sudo)	Selon distribution

Ces trois commandes s'appuient sur la configuration déclarée dans le fichier `/etc/network/interfaces` (Debian et dérivés). Contrairement à `ip link set`, elles appliquent l'ensemble de la configuration de l'interface : adressage, passerelle, routes et scripts associés.

COMMANDE	RÔLE
`ifup`	Active l'interface et applique sa configuration : elle peut alors émettre et recevoir des données
`ifdown`	Désactive l'interface : elle ne transmet ni ne reçoit plus aucune donnée
`ifquery`	Analyse et affiche la configuration déclarée d'une interface ou son état courant

bash

```
$ sudo ifup eth0
$ sudo ifdown eth0
# Configuration déclarée pour eth0
$ ifquery eth0
# Interfaces actuellement activées par ifupdown
$ ifquery --state
```

Exemple de déclaration d'une adresse statique dans `/etc/network/interfaces` :

/etc/network/interfaces

```
# Interface activée automatiquement au démarrage
auto eth0
iface eth0 inet static
    address 192.168.1.50/24
    gateway 192.168.1.1
```

> **SELON LA DISTRIBUTION**
> Sur Ubuntu (Netplan) et sur les distributions Red Hat récentes (NetworkManager), ces commandes peuvent être absentes ou n'être que des enveloppes autour de NetworkManager. Utilisez alors `nmcli connection up` et `nmcli connection down` (fiche 09).

COMMANDE 04 · CONFIGURATION

04 ethtool — Inspecter et régler la carte réseau

PAQUET	PRIVILÈGES	STATUT
ethtool	root recommandé (lecture complète et modification)	Actif

`ethtool` interroge et modifie les paramètres du contrôleur d'interface réseau (NIC) et de son pilote : vitesse, duplex, auto-négociation, détection du lien, statistiques matérielles et fonctions de déchargement (*offload*). C'est l'outil idéal pour diagnostiquer un problème de couche physique : câble défectueux, lien négocié à 100 Mb/s au lieu de 1 Gb/s, port mal identifié…

bash

```
$ sudo ethtool enp1s0
Settings for enp1s0:
        Supported link modes:   10baseT/Half 10baseT/Full
                                100baseT/Half 100baseT/Full
                                1000baseT/Full

        Speed: 1000Mb/s
        Duplex: Full
        Auto-negotiation: on
        Port: Twisted Pair
        Link detected: yes
```

COMMANDE	RÉSULTAT
`ethtool -i enp1s0`	Pilote, version et firmware de la carte
`ethtool -S enp1s0`	Statistiques détaillées du pilote (erreurs, pertes, CRC)
`ethtool -k enp1s0`	Fonctions de déchargement (offload) actives
`ethtool -p enp1s0 10`	Fait clignoter la LED du port pendant 10 s pour l'identifier physiquement
`ethtool -s enp1s0 speed 1000 duplex full autoneg off`	Forcer la vitesse et le mode duplex

DIAGNOSTIC RAPIDE

La ligne `Link detected: yes/no` confirme immédiatement si le lien physique est établi : c'est la première vérification à faire lorsqu'une machine « ne voit plus le réseau ».

COMMANDE 05 · DÉPANNAGE

05 ping — Tester la connectivité

PAQUET	PRIVILÈGES	STATUT
iputils-ping (Debian/Ubuntu) · iputils (RHEL/Fedora)	Utilisateur	Actif

`ping` (*Packet Internet Groper*) vérifie qu'un hôte est joignable, sur un réseau local (LAN) comme sur un réseau étendu (WAN). Il envoie des requêtes ICMP *Echo Request* et mesure le temps d'aller-retour (RTT) ainsi que le taux de perte. C'est le test de connectivité le plus élémentaire : il suffit de lui fournir une adresse IP ou un nom d'hôte.



Figure 2 — *Une requête ICMP Echo et sa réponse : le délai entre les deux est le RTT.*

```bash
# Envoyer 4 paquets puis s'arrêter
$ ping -c 4 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=0.412 ms
64 bytes from 192.168.1.1: icmp_seq=2 ttl=64 time=0.388 ms
64 bytes from 192.168.1.1: icmp_seq=3 ttl=64 time=0.405 ms
64 bytes from 192.168.1.1: icmp_seq=4 ttl=64 time=0.397 ms

--- 192.168.1.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3052ms
rtt min/avg/max/mdev = 0.388/0.400/0.412/0.009 ms
```

OPTION	EFFET
`-c <n>`	Nombre de paquets à envoyer avant de s'arrêter
`-i <s>`	Intervalle entre deux paquets (1 seconde par défaut)
`-s <octets>`	Taille des données envoyées (tests de MTU et de fragmentation)
`-W <s>`	Délai d'attente maximal d'une réponse
`-I <interface>`	Interface ou adresse source à utiliser
`-4 / -6`	Forcer IPv4 ou IPv6
`-q`	Mode silencieux : seul le résumé final est affiché

BON À SAVOIR

Sans l'option `-c`, ping s'exécute en continu : interrompez-le avec Ctrl + C. Une absence de réponse ne prouve pas que l'hôte est hors ligne : de nombreux pare-feu bloquent volontairement l'ICMP.

COMMANDE 06 · DÉPANNAGE

06 traceroute — Tracer le chemin des paquets

PAQUET	PRIVILÈGES	STATUT
traceroute	Utilisateur (root pour les sondes -I et -T)	Actif

`traceroute` affiche chaque routeur (saut) traversé entre votre machine et une destination, avec le temps de réponse de chacun. Il exploite le champ TTL des paquets IP : chaque routeur qui ramène le TTL à zéro renvoie un message ICMP et révèle ainsi sa présence. C'est l'outil qui permet de localiser l'endroit précis où une connexion se dégrade ou s'interrompt.

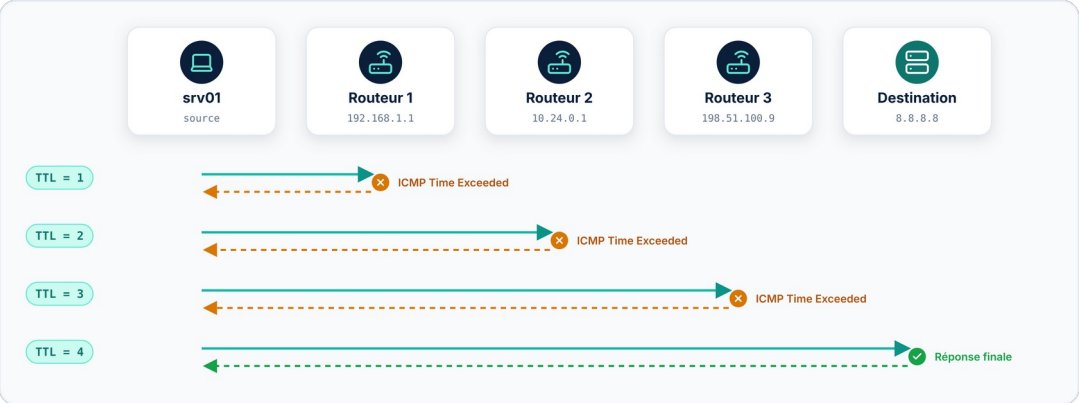


Figure 3 — *Principe de traceroute : le TTL augmente d'une unité à chaque série de sondes.*

```bash
$ traceroute -n 8.8.8.8
traceroute to 8.8.8.8 (8.8.8.8), 30 hops max, 60 byte packets
 1  192.168.1.1     0.487 ms  0.277 ms  0.269 ms
 2  10.24.0.1       1.846 ms  1.631 ms  1.553 ms
 3  * * *
 4  198.51.100.9    3.762 ms  3.683 ms  3.577 ms
 5  8.8.8.8         4.666 ms  4.869 ms  4.245 ms
```

Chaque ligne correspond à un saut ; les trois valeurs sont les temps de réponse de trois sondes successives. Les astérisques `* * *` signifient que le routeur n'a pas répondu dans le délai imparti — souvent à cause d'un filtrage ICMP — sans forcément indiquer une panne.

OPTION	EFFET
`-n`	Ne pas résoudre les noms d'hôtes (affichage plus rapide)
`-I`	Utiliser des sondes ICMP au lieu de paquets UDP
`-T -p 443`	Sondes TCP SYN vers un port : traverse mieux les pare-feu
`-m <n>`	Nombre maximal de sauts (30 par défaut)

> **ALTERNATIVE**
> `tracepath` fonctionne de manière similaire sans privilèges root et détermine en plus la MTU du chemin (*Path MTU*).

COMMANDE 07 · DÉPANNAGE

07 mtr — Diagnostic réseau en temps réel

PAQUET	PRIVILÈGES	STATUT
mtr (mtr-tiny : version console)	Utilisateur	Actif

`mtr` (*My Traceroute*) combine les fonctions de `ping` et de `traceroute` dans un seul outil moderne. Il sonde en continu chaque saut du chemin et met à jour en temps réel le taux de perte et la latence de chacun, jusqu'à ce que vous quittiez le programme avec la touche `q`. C'est l'outil privilégié pour identifier un lien instable ou une perte de paquets intermittente.

```bash
# Mode interactif (nom d'hôte ou adresse IP)
$ mtr example.com
# Mode rapport : 10 cycles, sortie large, puis arrêt automatique
$ mtr -rw -c 10 example.com
HOST: srv01                Loss%   Snt   Last   Avg  Best  Wrst StDev
  1.|-- 192.168.1.1         0.0%    10    0.5   0.6   0.4   1.7   0.4
  2.|-- 10.24.0.1           0.0%    10    1.9   1.5   0.8   7.3   1.9
  3.|-- ???                100.0    10    0.0   0.0   0.0   0.0   0.0
  4.|-- 198.51.100.9        0.0%    10    9.1   8.2   7.2  11.4   1.2
  5.|-- 203.0.113.50        0.0%    10   12.0  12.3  11.8  13.1   0.4
```

COLONNE	SIGNIFICATION
Loss%	Pourcentage de paquets perdus sur ce saut
Snt	Nombre de paquets envoyés
Last / Avg	Dernière latence mesurée / latence moyenne (en ms)
Best / Wrst	Meilleure et pire latence observées
StDev	Écart-type : une valeur élevée signale une latence instable (gigue)

INTERPRÉTER LES PERTES

Une perte sur un saut intermédiaire qui ne se répercute pas sur les sauts suivants vient généralement d'un routeur qui limite ses réponses ICMP. Seule la perte observée jusqu'à la destination finale est réellement significative.

COMMANDE 08 · CONFIGURATION

08 route — Gérer la table de routage

PAQUET	PRIVILÈGES	STATUT
net-tools	Lecture : utilisateur · Modification : root	Obsolète

`route` affiche et modifie la table de routage IP du noyau. Elle sert principalement à définir des routes statiques vers un hôte ou un réseau, via une passerelle et une interface données.

```bash
# Table de routage avec adresses numériques
$ route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0         192.168.1.1     0.0.0.0         UG    100    0        0 enp1s0
192.168.1.0     0.0.0.0         255.255.255.0   U     100    0        0 enp1s0
```

Le drapeau `U` signale une route active et `G` une route qui passe par une passerelle. La destination `0.0.0.0` correspond à la **route par défaut**.

```bash
# Ajouter une passerelle par défaut
$ sudo route add default gw 192.168.1.1
# Ajouter une route vers un réseau via une passerelle et une interface
$ sudo route add -net 10.10.0.0 netmask 255.255.255.0 gw 192.168.1.254 dev eth0
# Supprimer cette route
$ sudo route del -net 10.10.0.0 netmask 255.255.255.0
```

AVEC ROUTE (HISTORIQUE)	AVEC IP (MODERNE)
`route -n`	`ip route show`
`route add default gw 192.168.1.1`	`ip route add default via 192.168.1.1`
`route add -net 10.10.0.0 netmask 255.255.255.0 gw 192.168.1.254`	`ip route add 10.10.0.0/24 via 192.168.1.254`
`route del -net 10.10.0.0 netmask 255.255.255.0`	`ip route del 10.10.0.0/24`

COMMANDE 09 · CONFIGURATION

09 nmcli — Piloter NetworkManager

PAQUET	PRIVILÈGES	STATUT
network-manager (Debian/Ubuntu) · NetworkManager (RHEL/Fedora)	Lecture : utilisateur · Modification : root	Recommandé

`nmcli` est l'interface en ligne de commande de NetworkManager, le service de gestion réseau utilisé par défaut sur RHEL, Fedora, Rocky Linux, AlmaLinux et la plupart des postes de travail. Simple à utiliser et facile à scripter, il permet de consulter l'état du réseau, de créer et modifier des connexions et de gérer le Wi-Fi. Contrairement à `ip`, **les modifications sont persistantes**.

bash

```
# État des périphériques réseau
$ nmcli device status
DEVICE   TYPE      STATE         CONNECTION
enp1s0   ethernet  connected     Wired connection 1
wlp2s0   wifi      disconnected  --
# Toutes les connexions (profils), puis seulement les actives
$ nmcli connection show
$ nmcli connection show --active
```

Configurer une adresse IP statique

bash

```
$ sudo nmcli connection modify "Wired connection 1" \
      ipv4.method manual \
      ipv4.addresses 192.168.1.50/24 \
      ipv4.gateway 192.168.1.1 \
      ipv4.dns "1.1.1.1 9.9.9.9"
# Appliquer la nouvelle configuration
$ sudo nmcli connection up "Wired connection 1"
```

Gérer le Wi-Fi

bash

```
$ nmcli device wifi list
$ sudo nmcli device wifi connect "MonReseau" password "MotDePasse"
```

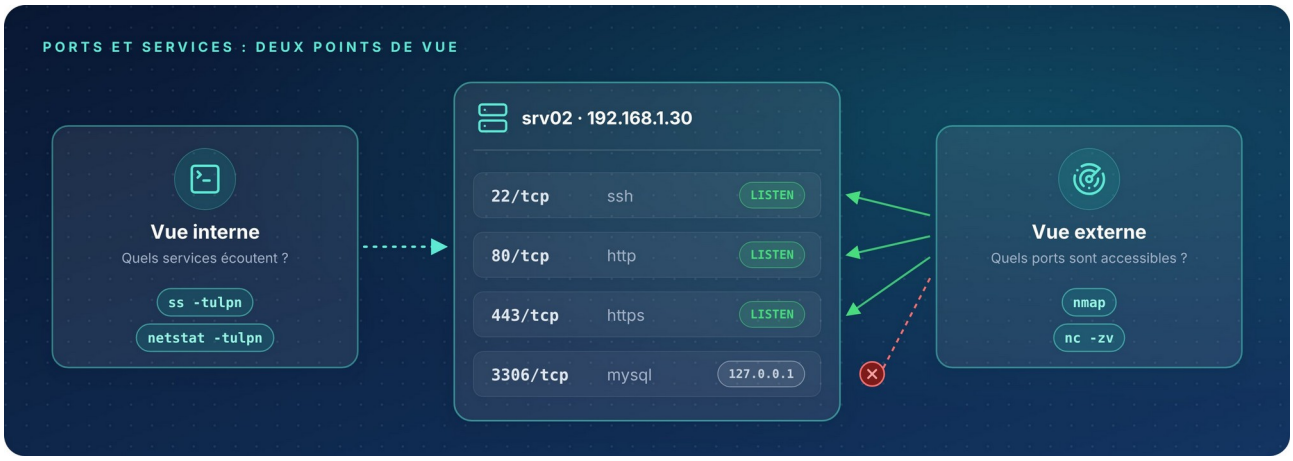
ASTUCE

Les mots-clés s'abrègent (`nmcli d s` pour `nmcli device status`) et la touche Tab complète les noms de connexions et de paramètres. `nmcli device show enp1s0` affiche d'un coup l'adresse, la passerelle et les DNS d'une interface.



Analyse du réseau et des performances

Une fois la connectivité établie, il faut savoir quels services écoutent, quelles connexions sont ouvertes et quels ports sont accessibles. Ces outils sont au cœur du débogage des services réseau et des audits de sécurité.



DANS CETTE PARTIE

10	**netstat**	Connexions, ports et statistiques
11	**ss**	Statistiques des sockets
12	**nc (netcat)**	Tester des ports, transférer des données
13	**nmap**	Découverte d'hôtes et scan de ports

COMMANDE 10 · ANALYSE

10 netstat — Connexions, ports et statistiques

PAQUET	PRIVILÈGES	STATUT
net-tools	Utilisateur (root pour voir tous les processus)	Obsolète

`netstat` affiche des informations sur le sous-système réseau de Linux : connexions actives, ports en écoute, table de routage, statistiques par interface et par protocole. Il a longtemps été l'outil de base pour le dépannage et l'analyse des performances, en particulier pour vérifier quel programme écoute sur quel port.

```bash
# Ports TCP/UDP en écoute, numériques, avec le programme associé
$ sudo netstat -tulpn
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address    Foreign Address  State   PID/Program name
tcp        0      0 0.0.0.0:22       0.0.0.0:*        LISTEN  972/sshd
tcp        0      0 127.0.0.1:3306   0.0.0.0:*        LISTEN  1053/mysqld
tcp6       0      0 :::80            :::*             LISTEN  990/httpd
udp        0      0 0.0.0.0:68       0.0.0.0:*                811/dhclient
```

OPTION	EFFET
`-t / -u`	Afficher les connexions TCP / UDP
`-l`	Afficher uniquement les sockets en écoute
`-p`	Afficher le PID et le nom du programme (root requis pour tous les voir)
`-n`	Adresses et ports numériques, sans résolution de noms
`-r`	Table de routage du noyau (équivalent de route -n)
`-i`	Statistiques par interface
`-s`	Statistiques par protocole (IP, ICMP, TCP, UDP)

OUTIL OBSOLÈTE

Comme `ifconfig` et `route`, `netstat` fait partie de net-tools. Son remplaçant moderne est `ss` (fiche 11), plus rapide et plus précis, dont les options principales sont identiques.

COMMANDE 11 · ANALYSE

11 ss — Statistiques des sockets

PAQUET	PRIVILÈGES	STATUT
iproute2 (Debian/Ubuntu) · iproute (RHEL/Fedora)	Utilisateur (root pour voir tous les processus)	Recommandé

`ss` (*socket statistics*) interroge directement le noyau pour lister les sockets. Il affiche des informations similaires à `netstat`, mais plus rapidement sur les serveurs chargés, et donne davantage de détails sur l'état des connexions TCP (minuteurs, mémoire, fenêtre de congestion).

bash

```
# Ports TCP en écoute, sortie numérique
$ ss -tln
State   Recv-Q  Send-Q   Local Address:Port   Peer Address:Port
LISTEN  0       128            0.0.0.0:22          0.0.0.0:*
LISTEN  0       80           127.0.0.1:3306        0.0.0.0:*
LISTEN  0       511                *:80                *:*
```

bash

```
# Ports TCP et UDP en écoute, avec les processus
$ sudo ss -tulpn
# Toutes les sockets TCP (en écoute et établies)
$ ss -ta
# Connexions TCP avec leurs minuteurs
$ ss -to
# Résumé statistique de toutes les sockets
$ ss -s
```

Filtrer avec précision

`ss` intègre un langage de filtrage par état et par port, très pratique pour isoler un trafic précis :

bash

```
# Connexions établies vers des serveurs HTTPS
$ ss -tn state established '( dport = :443 )'
# Qui est connecté à mon serveur SSH ?
$ ss -tn state established '( sport = :22 )'
```

> LE RÉFLEXE À RETENIR
>
> `sudo ss -tulpn` répond en une seule ligne à la question la plus fréquente du dépannage : « quel service écoute sur quel port, et sur quelle adresse ? ».

COMMANDE 12 · ANALYSE

12 nc (netcat) — Le couteau suisse du réseau

PAQUET	PRIVILÈGES	STATUT
netcat-openbsd (Debian/Ubuntu) · nmap-ncat (RHEL/Fedora)	Utilisateur (root pour les ports < 1024)	Actif

`nc` (netcat) lit et écrit des données à travers des connexions TCP, UDP ou des sockets UNIX. Il permet d'ouvrir une connexion, d'écouter sur un port, de vérifier l'accessibilité d'un service distant, de scanner quelques ports, de servir de relais TCP simple ou encore de transférer des fichiers entre deux machines.

Tester des ports distants

```bash
# -z : sonder sans envoyer de données · -v : mode verbeux
$ nc -zv srv02.example.lan 22 80 443
Connection to srv02.example.lan 22 port [tcp/ssh] succeeded!
Connection to srv02.example.lan 80 port [tcp/http] succeeded!
nc: connect to srv02.example.lan port 443 (tcp) failed: Connection refused
# Une plage de ports
$ nc -zv srv02.example.lan 20-90
# Port source 3000, délai d'attente de 10 s, vers le port 5000
$ nc -p 3000 -w 10 srv02.example.lan 5000
```

Transférer un fichier

```bash
# Machine réceptrice : écouter sur le port 5000
$ nc -l 5000 > sauvegarde.tar.gz
# Machine émettrice : envoyer le fichier
# (-N ferme la connexion en fin de fichier, variante OpenBSD)
$ nc -N srv02.example.lan 5000 < sauvegarde.tar.gz
```

> **ATTENTION**
> Les variantes de netcat (OpenBSD, traditionnelle, `ncat` de Nmap) n'ont pas exactement les mêmes options : vérifiez avec `man nc`. Netcat transmet les données **en clair** : pour des données sensibles, préférez `scp` ou `rsync` sur SSH.

COMMANDE 13 · ANALYSE · SÉCURITÉ

13 nmap — Cartographier le réseau et auditer les ports

PAQUET	PRIVILÈGES	STATUT
nmap	Utilisateur (root pour scans SYN, UDP et détection d'OS)	Actif

`nmap` (*Network Mapper*) est un scanner réseau puissant et extrêmement polyvalent. Il découvre les hôtes actifs d'un réseau, identifie les ports ouverts et les services qui y répondent, détecte les versions logicielles et le système d'exploitation. C'est un outil central pour l'inventaire, l'audit réseau et le durcissement de la sécurité.

bash

```
$ nmap 192.168.1.30
Starting Nmap 7.94 ( https://nmap.org )
Nmap scan report for 192.168.1.30
Host is up (0.00031s latency).
Not shown: 996 closed tcp ports (reset)
PORT     STATE SERVICE
22/tcp   open  ssh
80/tcp   open  http
443/tcp  open  https
3306/tcp open  mysql

Nmap done: 1 IP address (1 host up) scanned in 0.18 seconds
```

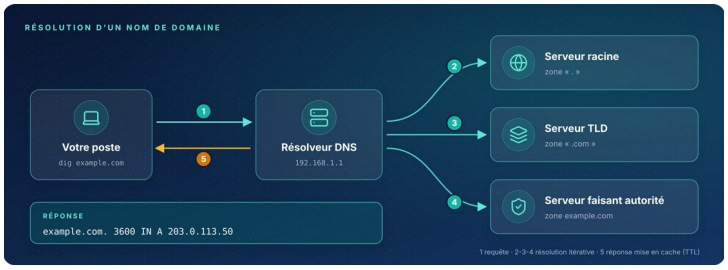
COMMANDE	USAGE
`nmap -sn 192.168.1.0/24`	Découvrir les hôtes actifs d'un réseau (sans scan de ports)
`nmap -p 22,80,443 <hôte>`	Scanner des ports précis
`nmap -p- <hôte>`	Scanner les 65 535 ports TCP
`nmap -sV <hôte>`	Identifier les services et leurs versions
`sudo nmap -O <hôte>`	Détecter le système d'exploitation
`sudo nmap -sU -p 53,123 <hôte>`	Scanner des ports UDP
`sudo nmap -A <hôte>`	Scan complet : OS, versions, scripts et traceroute
`nmap --script vuln <hôte>`	Scripts NSE de détection de vulnérabilités connues

> **CADRE LÉGAL**
> Ne scannez que les systèmes dont vous avez la responsabilité ou pour lesquels vous disposez d'une **autorisation écrite**. Un scan non autorisé peut être assimilé à une tentative d'intrusion et engager votre responsabilité.



Résolution DNS

Une grande partie des incidents réseau sont en réalité des problèmes de résolution de noms. Ces trois utilitaires interrogent les serveurs DNS pour traduire un nom en adresse IP (et inversement) et inspecter les différents types d'enregistrements d'une zone.



DANS CETTE PARTIE

14	**host**	Résolution DNS simple
15	**dig**	Interrogation DNS détaillée
16	**nslookup**	Interrogation DNS directe ou interactive

REPÈRES

ENREGISTREMENT	CONTENU
A / AAAA	Adresse IPv4 / adresse IPv6 associée à un nom
CNAME	Alias pointant vers un autre nom
MX	Serveurs de messagerie du domaine, avec leur priorité
NS	Serveurs de noms faisant autorité pour la zone
TXT	Texte libre : SPF, DKIM, vérifications de propriété
PTR	Résolution inverse : de l'adresse IP vers le nom

COMMANDE 14 · DNS

14 host — Résolution DNS rapide

PAQUET	PRIVILÈGES	STATUT
bind9-dnsutils (Debian/Ubuntu) · bind-utils (RHEL/Fedora)	Utilisateur	Actif

`host` est l'utilitaire de recherche DNS le plus simple : il traduit un nom d'hôte en adresses IP et une adresse IP en nom d'hôte, avec une sortie concise et lisible. Idéal pour une vérification rapide.

bash

```
$ host example.com
example.com has address 203.0.113.50
example.com has IPv6 address 2001:db8::50
example.com mail is handled by 10 mail.example.com.
# Un type d'enregistrement précis
$ host -t MX example.com
# Résolution inverse (adresse IP vers nom)
$ host 8.8.8.8
8.8.8.8.in-addr.arpa domain name pointer dns.google.
# Interroger un serveur DNS particulier
$ host example.com 1.1.1.1
```

OPTION	EFFET
`-t <type>`	Type d'enregistrement à interroger (A, AAAA, MX, NS, TXT…)
`-a`	Toutes les informations disponibles (équivalent de ANY)
`-4 / -6`	Utiliser IPv4 ou IPv6 pour joindre le serveur DNS
`-W <s>`	Délai d'attente maximal de la réponse

COMMANDE 15 · DNS

15 dig — L'outil DNS de référence

PAQUET	PRIVILÈGES	STATUT
bind9-dnsutils (Debian/Ubuntu) · bind-utils (RHEL/Fedora)	Utilisateur	Recommandé

`dig` (*Domain Information Groper*) interroge les serveurs DNS et affiche la réponse de façon détaillée : en-tête, question, réponse, temps de requête et serveur utilisé. Il permet de consulter tous les types d'enregistrements (A, CNAME, MX, NS…) et sa précision en fait l'outil privilégié pour diagnostiquer une zone DNS.

```bash
$ dig example.com
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 23083
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; QUESTION SECTION:
;example.com.                   IN      A

;; ANSWER SECTION:
example.com.            3600    IN      A       203.0.113.50

;; Query time: 4 msec
;; SERVER: 192.168.1.1#53(192.168.1.1) (UDP)
```

Points clés : `status: NOERROR` indique une réponse valide (`NXDOMAIN` : le nom n'existe pas ; `SERVFAIL` : échec du serveur). Dans la section ANSWER, `3600` est la durée de vie (TTL) de l'enregistrement en cache, en secondes.

COMMANDE	USAGE
`dig +short example.com`	Afficher uniquement le résultat
`dig MX example.com`	Interroger un type d'enregistrement
`dig @1.1.1.1 example.com`	Utiliser un serveur DNS précis
`dig -x 8.8.8.8`	Résolution inverse
`dig +trace example.com`	Suivre la résolution depuis les serveurs racine
`dig +noall +answer example.com`	N'afficher que la section réponse

> **MÉTHODE**
> Comparer `dig @<dns-interne> nom` et `dig @1.1.1.1 nom` permet de savoir immédiatement si un problème vient de votre résolveur interne ou de la zone publique.

COMMANDE 16 · DNS

16 nslookup — Interroger le DNS en mode direct ou interactif

PAQUET	PRIVILÈGES	STATUT
bind9-dnsutils (Debian/Ubuntu) · bind-utils (RHEL/Fedora)	Utilisateur	Actif

`nslookup` est un utilitaire très répandu — il existe aussi sous Windows — qui interroge les serveurs DNS soit directement (une requête en ligne de commande), soit en mode interactif (une invite dans laquelle on enchaîne les requêtes). Il permet notamment de trouver l'enregistrement A (adresse IP) d'un domaine ou d'effectuer une résolution inverse.

```bash
$ nslookup example.com
Server:         192.168.1.1
Address:        192.168.1.1#53

Non-authoritative answer:
Name:   example.com
Address: 203.0.113.50
```

« *Non-authoritative answer* » signifie que la réponse provient du cache d'un résolveur, et non directement du serveur faisant autorité pour la zone.

```bash
# Résolution inverse
$ nslookup 8.8.8.8
# Type d'enregistrement et serveur DNS précis
$ nslookup -type=MX example.com 1.1.1.1
# Mode interactif
$ nslookup
> set type=NS
> example.com
> exit
```

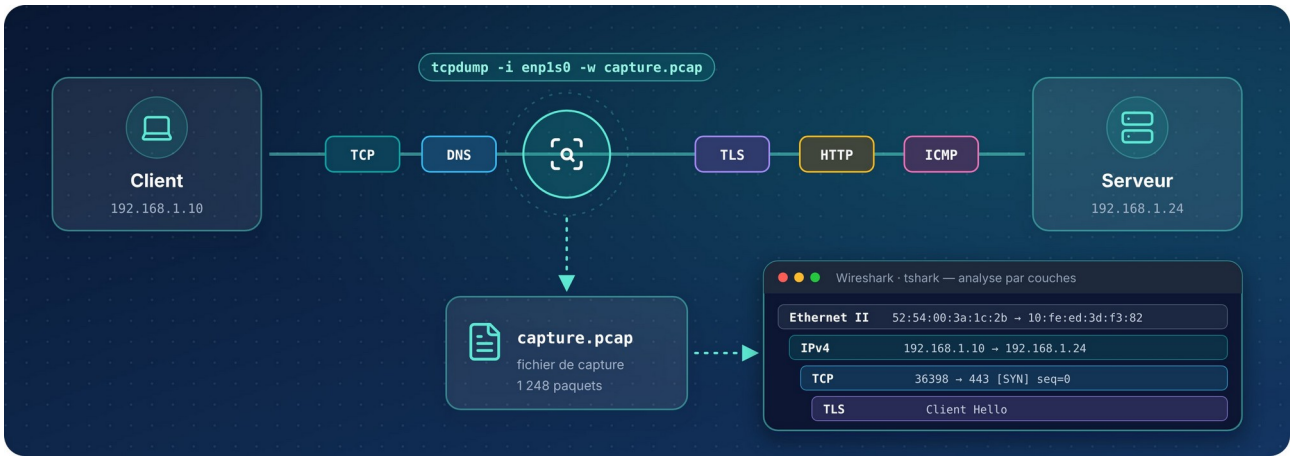
host, dig ou nslookup : lequel choisir ?

BESOIN	OUTIL CONSEILLÉ
Vérification rapide d'un nom ou d'une adresse	`host`
Diagnostic détaillé, scripts (`+short`), suivi de résolution	`dig`
Habitudes Windows, enchaînement de requêtes interactives	`nslookup`



Capture et analyse de paquets

Lorsque les autres outils ne suffisent plus, il faut observer le trafic lui-même. Les analyseurs de paquets capturent ce qui transite réellement sur une interface : ils permettent de comprendre un échange protocolaire dans ses moindres détails.



DANS CETTE PARTIE

17	`tcpdump`	Capture de paquets en console
18	`Wireshark`	Analyse graphique des protocoles

COMMANDE 17 · CAPTURE

17 tcpdump — Capturer les paquets en console

PAQUET	PRIVILÈGES	STATUT
tcpdump	root (sudo)	Actif

`tcpdump` est l'analyseur de paquets en ligne de commande le plus utilisé. Il capture les paquets TCP/IP émis et reçus sur une interface donnée, les affiche en temps réel ou les enregistre dans un fichier au format pcap pour une analyse ultérieure, par exemple dans Wireshark.

bash

```
# Lister les interfaces disponibles
$ sudo tcpdump -D
# Capturer sur une interface, sans résolution de noms ni de ports
$ sudo tcpdump -nn -i enp1s0
listening on enp1s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
09:35:40.287439 IP 192.168.1.24.22 > 192.168.1.10.36398: Flags [P.], ..., length 196
09:35:40.287655 IP 192.168.1.10.36398 > 192.168.1.24.22: Flags [.], ..., length 0
```

bash

```
# Capturer seulement 5 paquets
$ sudo tcpdump -c 5 -i enp1s0
# Enregistrer la capture dans un fichier
$ sudo tcpdump -i enp1s0 -w capture.pcap
# Relire un fichier de capture
$ tcpdump -nn -r capture.pcap
```

Filtres de capture (syntaxe BPF)

FILTRE	EFFET
`host 192.168.1.10`	Trafic vers ou depuis cet hôte
`src 192.168.1.10 / dst …`	Selon l'adresse source ou destination
`port 53`	Trafic sur le port 53 (DNS)
`tcp port 443`	Trafic TCP sur le port 443 (HTTPS)
`net 10.0.0.0/8`	Trafic d'un réseau entier
`icmp`	Uniquement l'ICMP (ping, traceroute)

bash

```
# Les filtres se combinent avec and, or et not
$ sudo tcpdump -nn -i enp1s0 'tcp port 443 and host 192.168.1.10'
```

CONFIDENTIALITÉ

Une capture peut contenir des données sensibles (identifiants, cookies, contenus non chiffrés). Limitez la capture au strict nécessaire, protégez les fichiers .pcap et supprimez-les après analyse.

COMMANDE 18 · CAPTURE

18 Wireshark — Analyser les protocoles en mode graphique

PAQUET	PRIVILÈGES	STATUT
wireshark · tshark (Debian/Ubuntu) · wireshark-cli (RHEL)	Groupe wireshark ou root pour capturer	Actif

Wireshark est un analyseur de protocoles graphique, populaire, puissant et facile à prendre en main. Il capture le trafic en temps réel ou ouvre des fichiers pcap (produits par exemple par `tcpdump`), puis décode chaque paquet couche par couche. Administrateurs et ingénieurs réseau l'utilisent pour le dépannage, l'analyse de performances et les investigations de sécurité.

Fonctionnalités clés

- Décodage détaillé de chaque paquet, couche par couche (Ethernet, IP, TCP, HTTP…) ;
- Filtres d'affichage puissants pour isoler un flux précis ;
- « Suivre le flux TCP » pour reconstituer une conversation complète ;
- Statistiques : conversations, hiérarchie des protocoles, graphiques d'entrées/sorties ;
- Enregistrement et export des captures dans de nombreux formats.

Filtres d'affichage utiles

FILTRE	AFFICHE
`ip.addr == 192.168.1.10`	Les paquets impliquant cette adresse
`tcp.port == 443`	Le trafic sur le port 443
`dns`	Uniquement le trafic DNS
`http.request`	Les requêtes HTTP
`tcp.analysis.retransmission`	Les retransmissions TCP (signe de pertes)
`tcp.flags.reset == 1`	Les connexions réinitialisées (RST)

tshark : Wireshark en ligne de commande

```bash
# Capturer et afficher le trafic DNS
$ sudo tshark -i enp1s0 -f "port 53"
# Analyser un fichier pcap avec un filtre d'affichage
$ tshark -r capture.pcap -Y "tcp.analysis.retransmission"
# Capturer sans être root (puis se reconnecter)
$ sudo usermod -aG wireshark $USER
```

> **MÉTHODE EFFICACE**
> Sur un serveur sans interface graphique, capturez avec `tcpdump -w`, rapatriez le fichier .pcap sur votre poste, puis analysez-le confortablement dans Wireshark.



Surveillance du trafic

Mesurer la bande passante consommée en temps réel aide à repérer une saturation, un transfert anormal ou une interface sous-dimensionnée, et à estimer le débit réellement disponible.



DANS CETTE PARTIE

19 **bmon** Bande passante en temps réel

COMMANDE 19 · SURVEILLANCE

19 bmon — Surveiller la bande passante en temps réel

PAQUET	PRIVILÈGES	STATUT
bmon (Debian/Ubuntu · EPEL sur RHEL)	Utilisateur	Actif

`bmon` (*bandwidth monitor*) est un utilitaire de surveillance et de débogage réseau en mode texte pour les systèmes de type Unix. Il collecte les statistiques de chaque interface et les présente sous forme de compteurs et de graphiques directement dans le terminal. Fiable et léger, il permet de suivre la bande passante en temps réel et d'estimer le débit.

```bash
# Surveiller toutes les interfaces
$ bmon
# Surveiller uniquement une interface
$ bmon -p enp1s0
# Afficher les débits en bits plutôt qu'en octets
$ bmon -b
```

TOUCHE	ACTION DANS L'INTERFACE
↑ / ↓	Passer d'une interface à l'autre
d	Afficher ou masquer les statistiques détaillées
g	Afficher ou masquer les graphiques
i	Afficher des informations complémentaires sur l'interface
?	Afficher l'aide
q	Quitter

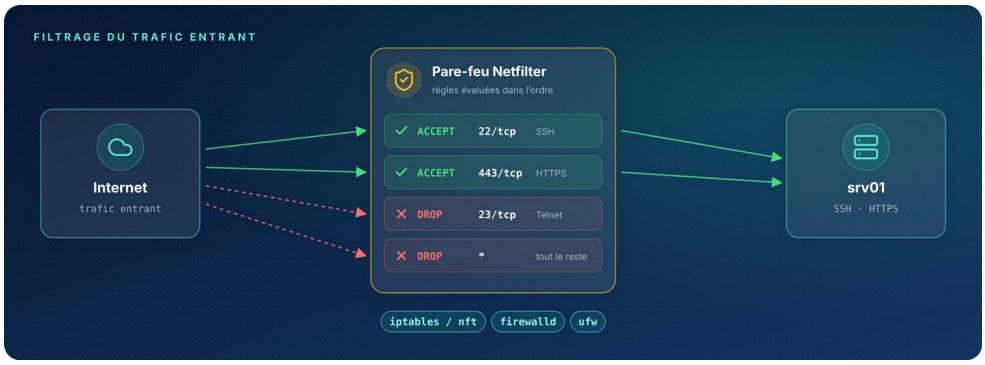
Outils complémentaires

OUTIL	SPÉCIALITÉ
`iftop`	Bande passante par connexion (qui consomme, vers où)
`nload`	Débits entrant et sortant d'une interface, sous forme de graphiques
`nethogs`	Bande passante consommée par processus
`vnstat`	Historique de consommation (horaire, journalier, mensuel)



Gestion du pare-feu

Sous Linux, le filtrage des paquets est assuré dans le noyau par le framework Netfilter. Les outils présentés ici sont des interfaces pour en définir les règles. Règle d'or : n'utilisez qu'un seul gestionnaire de pare-feu à la fois sur un même serveur.



DANS CETTE PARTIE

20	**iptables**	Règles Netfilter (filtrage, NAT)
21	**firewalld**	Pare-feu dynamique par zones
22	**ufw**	Pare-feu simplifié

REPÈRES

OUTIL	APPROCHE	DISTRIBUTIONS	NIVEAU
`iptables`	Chaînes et règles explicites	Toutes	Avancé
`firewalld`	Zones et services, rechargement à chaud	RHEL, Fedora, Rocky, Alma	Intermédiaire
`ufw`	Règles simplifiées, syntaxe lisible	Ubuntu, Debian	Débutant
`nft`	Successeur d'iptables, souvent utilisé en arrière-plan	Toutes (récentes)	Avancé

COMMANDE 20 · PARE-FEU

20 iptables — Le pare-feu Netfilter historique

PAQUET	PRIVILÈGES	STATUT
iptables (Debian/Ubuntu) · iptables-nft (RHEL/Fedora)	root (sudo)	Hérité

`iptables` permet de configurer, gérer et inspecter les tables de filtrage de paquets IPv4 et les règles NAT de Netfilter. Les règles sont organisées en tables (filter, nat, mangle) et en chaînes (INPUT, OUTPUT, FORWARD). On peut lister, ajouter, supprimer ou modifier des règles et consulter les compteurs associés à chacune.

Exemple : politique de base pour un serveur web

bash

```
# 1. Autoriser le trafic de retour et la boucle locale
$ sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
$ sudo iptables -A INPUT -i lo -j ACCEPT
# 2. Autoriser SSH, HTTP et HTTPS
$ sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
$ sudo iptables -A INPUT -p tcp -m multiport --dports 80,443 -j ACCEPT
# 3. Refuser tout le reste en entrée (politique par défaut)
$ sudo iptables -P INPUT DROP
# 4. Vérifier : règles avec compteurs et numéros de ligne
$ sudo iptables -L -n -v --line-numbers
```

OPTION	EFFET
`-A / -I <chaîne>`	Ajouter une règle en fin / en début de chaîne
`-D <chaîne> <n>`	Supprimer la règle numéro n
`-P <chaîne> <cible>`	Définir la politique par défaut (ACCEPT ou DROP)
`-p / --dport · -s`	Protocole, port de destination, adresse source
`-j <cible>`	Action : ACCEPT, DROP, REJECT, LOG…

Sauvegarder et restaurer

bash

```
$ sudo iptables-save | sudo tee /etc/iptables/rules.v4
$ sudo iptables-restore < /etc/iptables/rules.v4
# Debian/Ubuntu : rechargement automatique au démarrage
$ sudo apt install iptables-persistent
```

ADMINISTRATION À DISTANCE

Avant d'appliquer `-P INPUT DROP` sur un serveur distant, vérifiez que la règle autorisant SSH est en place, sinon vous perdrez l'accès. Gardez toujours une seconde session ouverte pendant vos tests.

COMMANDE 21 · PARE-FEU

21 firewalld — Le pare-feu dynamique par zones

PAQUET	PRIVILÈGES	STATUT
firewalld	root (sudo)	Actif

firewalld est un démon de gestion dynamique du pare-feu Netfilter, administré avec la commande `firewall-cmd`. Au lieu des chaînes d'iptables, il raisonne en **zones** (public, home, internal, trusted, drop…) auxquelles on associe interfaces, services et ports. Les règles se modifient à chaud, sans interrompre les connexions établies. C'est le pare-feu par défaut de RHEL, Fedora, Rocky Linux et AlmaLinux ; il s'appuie sur nftables dans les versions récentes.

bash

```
$ sudo firewall-cmd --state
running
$ sudo firewall-cmd --get-active-zones
public
  interfaces: enp1s0
# Tout ce qui est autorisé dans la zone par défaut
$ sudo firewall-cmd --list-all
```

Ouvrir des services et des ports

bash

```
# Autoriser HTTPS et le port 8080 de façon permanente, puis recharger
$ sudo firewall-cmd --zone=public --add-service=https --permanent
$ sudo firewall-cmd --zone=public --add-port=8080/tcp --permanent
$ sudo firewall-cmd --reload
# Retirer un service
$ sudo firewall-cmd --zone=public --remove-service=http --permanent
# Règle riche : SSH autorisé uniquement depuis le réseau local
$ RULE='rule family="ipv4" source address="192.168.1.0/24" service name="ssh" accept'
$ sudo firewall-cmd --permanent --add-rich-rule="$RULE"
```

RUNTIME OU PERMANENT ?

Sans `--permanent`, une modification s'applique immédiatement mais disparaît au prochain rechargement. Avec `--permanent`, elle est enregistrée mais ne s'applique qu'après `--reload`. Astuce : testez sans `--permanent`, puis validez avec `sudo firewall-cmd --runtime-to-permanent`.

UN SEUL GESTIONNAIRE

N'utilisez pas firewalld et le service iptables en même temps sur un même serveur : choisissez l'un ou l'autre.

COMMANDE 22 · PARE-FEU

22 ufw — Le pare-feu simplifié

PAQUET	PRIVILÈGES	STATUT
ufw	root (sudo)	Actif

UFW (*Uncomplicated Firewall*) est l'outil de configuration de pare-feu par défaut d'Ubuntu, également disponible sur Debian. Il simplifie l'écriture des règles Netfilter grâce à une syntaxe lisible : activer ou désactiver le pare-feu, ajouter, supprimer, modifier ou réinitialiser des règles devient très accessible.

bash

```
# Toujours autoriser SSH AVANT d'activer le pare-feu à distance
$ sudo ufw allow OpenSSH
$ sudo ufw allow 443/tcp
$ sudo ufw enable
$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
OpenSSH                    ALLOW IN    Anywhere
443/tcp                    ALLOW IN    Anywhere
OpenSSH (v6)               ALLOW IN    Anywhere (v6)
443/tcp (v6)               ALLOW IN    Anywhere (v6)
```

COMMANDE	EFFET
`sudo ufw allow 80/tcp`	Autoriser un port
`sudo ufw deny 23`	Bloquer un port
`sudo ufw allow from 192.168.1.0/24 to any port 22`	Autoriser un réseau sur un port précis
`sudo ufw limit ssh`	Limiter les tentatives de connexion (anti force brute)
`sudo ufw status numbered`	Lister les règles numérotées
`sudo ufw delete 3`	Supprimer la règle n° 3
`sudo ufw disable`	Désactiver le pare-feu
`sudo ufw reset`	Réinitialiser toutes les règles

POLITIQUE RECOMMANDÉE

`sudo ufw default deny incoming` et `sudo ufw default allow outgoing` : tout est bloqué en entrée, sauf ce que vous autorisez explicitement.

POUR CONCLURE

Conclusion

Ce guide a passé en revue les 22 outils et utilitaires en ligne de commande les plus utilisés pour la gestion du réseau sous Linux, classés par familles. Ils constituent le socle de compétences de tout administrateur système, et sont tout aussi utiles aux administrateurs et ingénieurs réseau à temps plein.



La maîtrise de ces commandes repose sur trois principes : **privilégier les outils modernes** (`ip`, `ss`, `nmcli`, nftables) tout en sachant lire les outils historiques encore présents sur de nombreux serveurs ; **procéder avec méthode**, de la couche physique jusqu'à l'application ; et **agir avec prudence**, en particulier sur les pare-feu et les serveurs distants.

Pour aller plus loin

- `man <commande>` : la documentation complète et officielle de chaque outil ;
- `<commande> --help` : un résumé rapide des options disponibles ;
- `tldr <commande>` : des exemples pratiques et concis (paquet tldr) ;
- Un laboratoire virtuel (VirtualBox, KVM, GNS3) pour s'exercer sans risque.

En résumé : la démarche de diagnostic

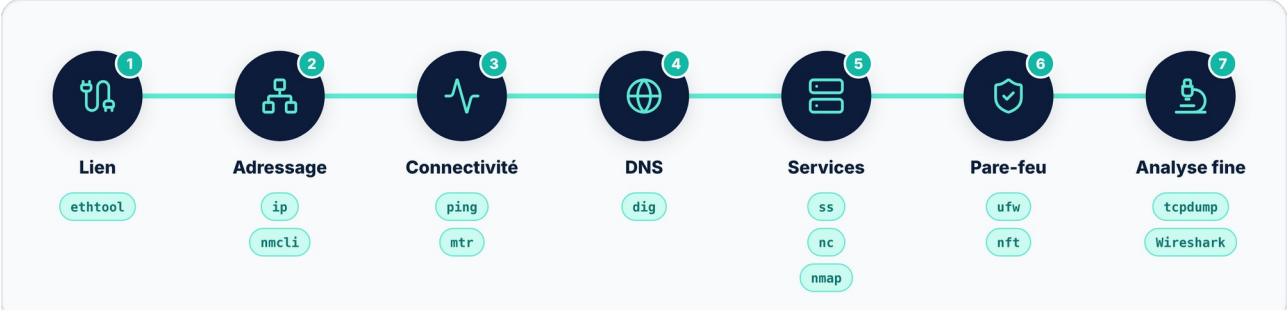


Figure 4 — *Les sept étapes d'un diagnostic réseau méthodique et les outils associés.*

RESSOURCES

Annexes

Ces annexes rassemblent les informations pratiques à garder sous la main : aide-mémoire, correspondances entre outils, méthode de diagnostic, installation, bonnes pratiques et glossaire.

A. Aide-mémoire : « Je veux… »

JE VEUX…	COMMANDE
Afficher mes adresses IP	`ip -br addr`
Connaître ma passerelle	`ip route`
Vérifier le lien physique	`sudo ethtool enp1s0`
Tester qu'une machine répond	`ping -c 4 <hôte>`
Trouver où la connexion se dégrade	`mtr -rw <hôte>`
Voir les ports en écoute	`sudo ss -tulpn`
Tester un port distant	`nc -zv <hôte> <port>`
Découvrir les machines du réseau	`nmap -sn 192.168.1.0/24`
Résoudre un nom de domaine	`dig +short <nom>`
Capturer le trafic d'une interface	`sudo tcpdump -nn -i enp1s0`
Surveiller la bande passante	`bmon`
Lister les connexions NetworkManager	`nmcli connection show`
Voir les règles du pare-feu	`sudo ufw status` · `sudo firewall-cmd --list-all` · `sudo nft list ruleset`

B. Correspondance net-tools → iproute2

Le paquet net-tools (`ifconfig`, `route`, `netstat`, `arp`) est remplacé par la suite iproute2. Ce tableau facilite la transition :

ANCIENNE COMMANDE	ÉQUIVALENT MODERNE	RÔLE
`ifconfig`	`ip addr`	Afficher les interfaces
`ifconfig eth0 up`	`ip link set eth0 up`	Activer une interface
`ifconfig eth0 mtu 9000`	`ip link set eth0 mtu 9000`	Modifier la MTU
`route -n`	`ip route`	Table de routage
`route add default gw <ip>`	`ip route add default via <ip>`	Passerelle par défaut
`arp -n`	`ip neigh`	Table ARP / voisins
`netstat -tulpn`	`ss -tulpn`	Ports en écoute
`netstat -i`	`ip -s link`	Statistiques d'interfaces
`netstat -r`	`ip route`	Table de routage

C. Méthode de diagnostic en 8 étapes

Face à un incident, procédez de la couche la plus basse vers la plus haute. Chaque étape validée élimine une catégorie de causes :

#	ÉTAPE	QUESTION	COMMANDES
1	Lien physique	L'interface est-elle active, le câble branché ?	`ip -br link · ethtool`
2	Adressage IP	L'adresse et le masque sont-ils corrects ?	`ip -br addr · nmcli device show`
3	Réseau local	La passerelle répond-elle ?	`ip route · ping <passerelle> · ip neigh`
4	Accès externe	Une adresse publique est-elle joignable ?	`ping 1.1.1.1 · mtr -rw 1.1.1.1`
5	Résolution DNS	Les noms sont-ils résolus ?	`dig <nom> · resolvectl status`
6	Service	Le port est-il ouvert et en écoute ?	`ss -tulpn · nc -zv · nmap -p`
7	Pare-feu	Un filtrage bloque-t-il le flux ?	`ufw status · firewall-cmd --list-all · nft list ruleset`
8	Analyse fine	Que se passe-t-il réellement sur le réseau ?	`tcpdump · Wireshark`

D. Installation des outils

COMMANDE(S)	DEBIAN / UBUNTU (APT)	RHEL / FEDORA (DNF)
ifconfig, route, netstat	net-tools	net-tools
ip, ss	iproute2	iproute
ifup, ifdown, ifquery	ifupdown	— (NetworkManager)
ethtool	ethtool	ethtool
ping	iputils-ping	iputils
traceroute	traceroute	traceroute
mtr	mtr / mtr-tiny	mtr
nmcli	network-manager	NetworkManager
nc	netcat-openbsd	nmap-ncat
nmap	nmap	nmap
host, dig, nslookup	bind9-dnsutils	bind-utils
tcpdump	tcpdump	tcpdump
wireshark, tshark	wireshark, tshark	wireshark, wireshark-cli
bmon	bmon	bmon (EPEL)
iptables	iptables	iptables-nft
firewall-cmd	firewalld	firewalld
ufw	ufw	ufw (EPEL)

bash

```
# Debian / Ubuntu
$ sudo apt update && sudo apt install -y iproute2 bind9-dnsutils mtr-tiny tcpdump nmap
# RHEL / Fedora / Rocky Linux / AlmaLinux
$ sudo dnf install -y iproute bind-utils mtr tcpdump nmap
```

E. Bonnes pratiques et sécurité

- **Gardez une seconde session SSH ouverte** lorsque vous modifiez le réseau ou le pare-feu d'un serveur distant, et testez avant de fermer la première.
- **Rendez vos changements persistants** avec l'outil de la distribution (`nmcli`, Netplan, `/etc/network/interfaces`) : `ip`, `ifconfig` et `route` ne survivent pas au redémarrage.
- **N'utilisez qu'un seul gestionnaire de pare-feu** par serveur (ufw, firewalld ou nftables).
- **Ne scannez que des systèmes autorisés** : `nmap` et `nc` sur un réseau tiers peuvent constituer une infraction.
- **Protégez les captures** : les fichiers .pcap peuvent contenir des mots de passe et des données personnelles.
- **Utilisez l'option `-n`** (`ss`, `netstat`, `traceroute`, `tcpdump`) pour accélérer les diagnostics en évitant les résolutions DNS.
- **Documentez chaque intervention** : commande exécutée, date, motif et moyen de retour arrière.

F. Sources et références

- Aaron Kili, « 22 commandes réseau Linux pour administrateur système », Tecmint, mis à jour le 23 juillet 2025 — tecmint.com/linux-networking-commands. Article dont ce guide reprend le plan et qu'il enrichit.
- Pages de manuel Linux : `ip(8)`, `ss(8)`, `nmcli(1)`, `ethtool(8)`, `tcpdump(8)`, `nmap(1)`, `dig(1)`, `firewall-cmd(1)`, `ufw(8)`, `iptables(8)`, `nft(8)`.
- Documentations officielles : nmap.org, wireshark.org, firewalld.org, netfilter.org.

G. Glossaire

TERME	DÉFINITION
ARP / NDP	Protocoles qui associent une adresse IP (IPv4 / IPv6) à une adresse MAC sur le réseau local
CIDR	Notation d'un réseau par préfixe, par exemple 192.168.1.0/24 (masque 255.255.255.0)
DNS	Système de noms de domaine : traduit les noms (example.com) en adresses IP
ICMP	Protocole de messages de contrôle et d'erreur d'IP, utilisé par ping et traceroute
Interface	Point de connexion réseau d'un système, physique (enp1s0) ou virtuel (lo, virbr0)
LAN / WAN	Réseau local / réseau étendu (Internet, liaisons entre sites)
MAC	Adresse matérielle unique d'une carte réseau (ex. 52:54:00:3a:1c:2b)
MTU	Unité de transmission maximale : taille maximale d'une trame (1500 octets en Ethernet)
NAT	Traduction d'adresses réseau, par exemple pour partager une adresse IP publique
Netfilter	Framework du noyau Linux qui filtre et transforme les paquets ; piloté par iptables, nftables, firewalld ou ufw
Passerelle	Routeur par lequel transitent les paquets destinés aux autres réseaux
pcap	Format de fichier standard des captures de paquets (tcpdump, Wireshark)
Port	Numéro (0 à 65 535) identifiant un service sur une machine : 22 = SSH, 443 = HTTPS
RTT	Round-Trip Time : temps d'aller-retour d'un paquet, mesuré par ping et mtr
Socket	Point de communication défini par une adresse IP, un port et un protocole
TTL	Time To Live : durée de vie d'un paquet IP (en sauts) ou d'un enregistrement DNS (en secondes)
Zone	Dans firewalld, ensemble de règles de confiance associé à des interfaces ou à des sources

DOCUMENT RÉDIGÉ ET MIS EN FORME PAR

Ngouye GNING

Version 1.0 · Octobre 2026